

TelemetriaMqtt v2

Relatório de Testes

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Catálogo de todos os testes automatizados do projeto até o momento: **61 testes** em 3 suítes independentes (firmware, backend, infraestrutura). Pra cada teste, uma frase explicando o que ele verifica e por que ele existe — não o nome da função, que já está listado, mas o raciocínio por trás. Este documento é gerado sob demanda e reflete o estado do projeto na data acima; para o número exato de testes a qualquer momento, rodar `pytest --collect-only` em cada suíte (ver `CLAUDE.md`).

Contents

1	Resumo	1
2	Filosofia de teste do projeto	2
3	Firmware	2
3.1	test/test_scaffolding/test_main.cpp	2
4	Backend — mock publisher (backend/mock_publisher/)	2
4.1	tests/test_signal_generator.py — geração de sinais (lógica pura)	2
4.2	tests/test_publisher.py — publicação (integração, client MQTT simulado)	3
4.3	tests/test_mock_publisher_main.py — CLI (parsing e wiring do client)	3
4.4	tests/test_mock_publisher_loop.py — loop de publicação (relógio/sleep injetáveis)	3
5	Backend — ingestão (backend/ingestion/)	4
5.1	tests/test_ingestion_transform.py — transformação payload → ponto (lógica pura)	4
5.2	tests/test_ingest.py — callback MQTT (integração, writer simulado)	4
5.3	tests/test_influx_writer.py — wiring do SDK InfluxDB (client mockado)	4
5.4	tests/test_ingestion_main.py — CLI (parsing, wiring do writer e do client MQTT)	5
6	Infraestrutura (infra/)	5
6.1	tests/test_mosquitto_acl.py — controle de acesso por tópico	5
6.2	tests/test_mosquitto_auth.py — autenticação	6
6.3	tests/test_generate_passwd.py — script de geração de credenciais	6
6.4	tests/test_docker_compose.py — docker-compose.yml real	6
6.5	tests/test_end_to_end.py — integração ponta a ponta (mock publisher → broker → ingestão)	7
7	O que ainda não é testado (decisão consciente, não descuido)	7

1 Resumo

Suíte	Testes	Como rodar
Firmware (firmware/)	1	<code>pio test -e native</code>
Backend (backend/)	43	<code>pytest</code> (dentro do venv de <code>backend/</code>)
Infra (infra/)	17	<code>pytest</code> (dentro do venv de <code>infra/</code> , exige Docker)
Total	61	

Todas as três suítes rodam automaticamente a cada `push/pull request` via GitHub Actions (`.github/workflows/ci.yml`).

2 Filosofia de teste do projeto

Três regras, definidas em `CLAUDE.md`, moldam como os testes abaixo foram escritos:

- **TDD:** testes escritos antes ou junto da implementação, nunca depois "pra ver se passa".
- **Testes são intocáveis depois de escritos:** nunca apagados ou alterados pra fazer passar — se um teste falha, conserta-se a implementação.
- **Lógica pura separada de integração:** cada módulo separa código sem I/O (fácil de testar exaustivamente) de código com rede/Docker/subprocessos (testado com dependências simuladas quando a infraestrutura real ainda não existe, e com a infraestrutura real assim que ela existe).

3 Firmware

3.1 `test/test_scaffolding/test_main.cpp`

Ambiente `native` do PlatformIO — roda no computador, sem hardware.

Teste	O que verifica e por quê
<code>test_native_test_harness_runs</code>	Prova (via <code>2+2==4</code>) que toda a cadeia de ferramentas do ambiente nativo funciona — download da biblioteca Unity, compilação, execução, relatório de resultado — antes de existir qualquer lógica real pra testar.

4 Backend — `mock publisher` (`backend/mock_publisher/`)

Simula o ESP32 publicando telemetria via MQTT, sem hardware.

4.1 `tests/test_signal_generator.py` — geração de sinais (lógica pura)

Teste	O que verifica e por quê
<code>test_signal_specs_cover_all_20_channels</code>	A tabela de especificações tem exatamente os 20 canais do carro, nem mais nem menos — evita esquecer ou duplicar um sinal ao editar.
<code>test_snapshot_has_all_channels_plus_timestamp</code>	O JSON de saída tem exatamente os 20 campos + <code>ts</code> , o schema que o resto do sistema depende.
<code>test_snapshot_values_are_within_declared_range</code>	Nenhum valor sintético (ex: RPM) sai da faixa física plausível declarada, em vários instantes de tempo.
<code>test_binary_signals_are_always_0_or_1</code>	Sinais digitais (<code>bombaInput</code> , <code>giratoriaInput</code>) nunca saem como float ou fora de <code>{0,1}</code> .
<code>test_slow_tier_value_is_held_constant_within_its_own_interval</code>	Um sinal de 2Hz mantém o mesmo valor dentro da própria janela de amostragem (comportamento de "degrau", não interpolação).
<code>test_slow_tier_value_changes_after_crossing_its_interval_boundary</code>	Complementa o teste anterior: ao cruzar a fronteira do intervalo, o valor muda — sem isso, uma implementação que sempre retorna o mesmo número fixo passaria no teste anterior.
<code>test_fast_tier_changes_more_often_than_slow_tier_over_one_second</code>	Um sinal de 20Hz mostra muito mais valores distintos que um de 2Hz no mesmo intervalo de 1s — pega uma troca acidental de frequências entre sinais.

Teste	O que verifica e por quê
<code>test_timestamp_is_monotonic_ms_integer</code>	<code>ts</code> é sempre inteiro em milissegundos, compatível com <code>millis()</code> do firmware real do ESP32.

4.2 `tests/test_publisher.py` — publicação (integração, client MQTT simulado)

Teste	O que verifica e por quê
<code>test_publish_snapshot_calls_client_publish_with_correct_topic</code>	Publica no tópico correto (<code>telemetria/esp32/data</code>), exatamente uma vez por snapshot.
<code>test_publish_snapshot_payload_is_valid_json_with_expected_keys</code>	O payload publicado é JSON válido, e <code>ts</code> bate com o tempo passado (2.5s → 2500ms).
<code>test_publish_snapshot_dry_run_does_not_touch_client</code>	Modo <code>-dry-run</code> nunca toca o client MQTT real — só usa a função <code>sink</code> alternativa.

4.3 `tests/test_mock_publisher_main.py` — CLI (parsing e wiring do client)

Teste	O que verifica e por quê
<code>test_parse_args_defaults</code>	Valores padrão dos argumentos de linha de comando (host, porta, TLS ligado, sem duração limite).
<code>test_parse_args_no_tls_flag</code>	A flag <code>-no-tls</code> é reconhecida pelo parser.
<code>test_build_client_returns_none_in_dry_run</code>	Modo <code>-dry-run</code> não abre nenhuma conexão de rede.
<code>test_build_client_enables_tls_by_default</code>	Teste de regressão: TLS fica ligado por padrão — esse era exatamente o comportamento que faltava e quebrou a validação manual no checkpoint 3.
<code>test_build_client_skips_tls_with_no_tls_flag</code>	<code>-no-tls</code> realmente desliga o TLS (o bug corrigido).
<code>test_build_client_sets_credentials_when_username_given</code>	Usuário/senha são repassados ao client quando fornecidos.
<code>test_build_client_skips_credentials_when_no_username</code>	Sem usuário informado, <code>username_pw_set</code> não é chamado (evita erro com credenciais parciais).
<code>test_build_client_connects_to_configured_host_and_port</code>	Conecta exatamente no host/porta configurados via CLI.

4.4 `tests/test_mock_publisher_loop.py` — loop de publicação (relógio/sleep injetáveis)

Teste	O que verifica e por quê
<code>test_run_publish_loop_stops_after_duration</code>	O loop respeita o limite de <code>-duration</code> e para no tick certo.
<code>test_run_publish_loop_passes_increasing_t_to_publish_snapshot</code>	O tempo (<code>t</code>) passado a cada chamada avança corretamente, tick a tick.

Teste	O que verifica e por quê
<code>test_run_publish_loop_forwards_dry_run_flag</code>	A flag <code>dry_run</code> é repassada corretamente pro <code>publish_snapshot</code> .
<code>test_run_publish_loop_passes_client_through</code>	O objeto <code>client</code> é repassado sem alteração.
<code>test_run_publish_loop_runs_forever_without_duration</code>	Sem <code>duration</code> , o loop não para sozinho (roda até ser interrompido externamente).

Os cinco testes acima usam um relógio e uma função de espera *falsos* (que avançam instantaneamente, sem esperar tempo real) — por isso a suíte inteira do backend roda em menos de 0,2 segundos, mesmo simulando um loop que normalmente rodaria minutos.

5 Backend — ingestão (backend/ingestion/)

Assina o broker MQTT e grava cada mensagem no InfluxDB.

5.1 `tests/test_ingestion_transform.py` — transformação payload → ponto (lógica pura)

Teste	O que verifica e por quê
<code>test_to_point_maps_all_20_numeric_fields</code>	Todo sinal declarado vira um campo do ponto gravado — nenhum esquecido.
<code>test_to_point_uses_measurement_name</code>	O nome da <i>measurement</i> é sempre "telemetry".
<code>test_to_point_uses_received_at_as_time_not_device_ts</code>	O timestamp do ponto é o horário de recebimento (parâmetro passado), não derivado do <code>ts</code> do payload — decisão de design central deste checkpoint (ver docs/checkpoint-04).
<code>test_to_point_keeps_device_ts_as_a_reference_field</code>	O <code>ts</code> original não é perdido: fica salvo como campo <code>device_ts_ms</code> .
<code>test_to_point_raises_on_missing_field</code>	Um payload incompleto levanta erro explícito, em vez de ser gravado parcialmente no banco.

5.2 `tests/test_ingest.py` — callback MQTT (integração, writer simulado)

Teste	O que verifica e por quê
<code>test_on_message_parses_valid_payload_and_calls_writer</code>	Uma mensagem MQTT válida chega, é transformada e o <i>writer</i> é chamado com o ponto correto.
<code>test_on_message_ignores_invalid_json</code>	JSON corrompido é descartado silenciosamente (do ponto de vista do writer) — não derruba o processo.
<code>test_on_message_ignores_payload_missing_fields</code>	Payload estruturalmente válido mas incompleto também é descartado, não gravado parcialmente.

5.3 `tests/test_influx_writer.py` — wiring do SDK InfluxDB (client mockado)

Teste	O que verifica e por quê
<code>test_writer_calls_write_api_with_correct_bucket_and_org</code>	<code>write_api.write(...)</code> é chamado com o bucket e a organização configurados.
<code>test_writer_builds_point_with_measurement_fields_and_time</code>	O <i>line protocol</i> do ponto construído tem measurement, campos e timestamp corretos.
<code>test_writer_skips_none_fields</code>	Um campo com valor <code>None</code> não é enviado ao SDK (que rejeitaria a chamada).

5.4 tests/test_ingestion_main.py — CLI (parsing, wiring do writer e do client MQTT)

Teste	O que verifica e por quê
<code>test_parse_args_defaults</code>	Valores padrão dos argumentos (host, porta, URL/org/bucket do InfluxDB).
<code>test_build_writer_returns_dry_run_writer_in_dry_run</code>	Em <code>-dry-run</code> , o writer só imprime, não grava.
<code>test_build_writer_builds_real_influx_client_when_not_dry_run</code>	Fora do <code>dry-run</code> , a URL/token/org/bucket são repassados corretamente pro SDK do InfluxDB (sem precisar de um banco real rodando).
<code>test_build_mqtt_client_enables_tls_by_default</code>	Teste de regressão , espelhando o mesmo bug já corrigido no <code>mock_publisher</code> : TLS precisa ficar ligado por padrão.
<code>test_build_mqtt_client_skips_tls_with_no_tls_flag</code>	<code>-no-tls</code> desliga o TLS de fato.
<code>test_build_mqtt_client_sets_credentials_when_username_given</code>	Usuário/senha repassados ao client MQTT.
<code>test_build_mqtt_client_subscribes_to_telemetry_topic</code>	Assina exatamente o tópico <code>telemetria/esp32/data</code> , com QoS 1.
<code>test_build_mqtt_client_wires_on_message_callback</code>	O callback de mensagem passado é de fato o que fica registrado no client.

6 Infraestrutura (infra/)

Broker MQTT (Mosquitto via Docker), `docker-compose.yml`, geração de credenciais, e integração ponta a ponta. Todos os testes desta seção sobem containers Docker **reais** — nenhum broker é simulado.

6.1 tests/test_mosquitto_acl.py — controle de acesso por tópico

Teste	O que verifica e por quê
<code>test_esp32_user_can_publish_to_its_topic</code>	O usuário do dispositivo (write-only) consegue publicar no próprio tópico.
<code>test_esp32_user_subscribes_but_never_receives_without_read_access</code>	Mesmo usuário consegue <i>assinar</i> o tópico (o Mosquitto aceita o SUBACK), mas nunca recebe nada — a ACL de leitura é aplicada na entrega, não na assinatura (descoberta documentada em <code>docs/checkpoint-03</code>).
<code>test_reader_can_subscribe_and_receive</code>	O usuário consumidor (read-only) recebe corretamente o que o dispositivo publica.

Teste	O que verifica e por quê
<code>test_reader_cannot_publish</code>	O mesmo usuário consumidor não consegue publicar (reason code de recusa).

6.2 tests/test_mosquitto_auth.py — autenticação

Teste	O que verifica e por quê
<code>test_correct_credentials_are_accepted</code>	Baseline positivo: credenciais corretas são aceitas — garante que os testes de recusa abaixo não passariam só porque o broker está fora do ar.
<code>test_wrong_password_is_rejected</code>	Usuário válido, senha errada, é recusado. A propriedade que motivou o checkpoint 3 inteiro (evitar repetir o vazamento do TCC antigo).
<code>test_unknown_username_is_rejected</code>	Usuário que não existe no <code>passwd</code> é recusado.
<code>test_anonymous_connection_is_rejected</code>	Conexão sem usuário/senha nenhum é recusada (<code>allow_anonymous false</code> funciona de verdade).

6.3 tests/test_generate_passwd.py — script de geração de credenciais

Teste	O que verifica e por quê
<code>test_creates_passwd_file_if_missing</code>	O script cria o arquivo <code>passwd</code> do zero se ele ainda não existir.
<code>test_adds_entry_for_the_given_username</code>	O usuário informado aparece corretamente no arquivo gerado.
<code>test_adding_a_second_user_preserves_the_first</code>	Adicionar um segundo usuário não apaga o primeiro (comportamento de <i>append</i> , não sobrescrita).
<code>test_rerunning_for_the_same_user_updates_not_duplicates</code>	Rodar de novo pro mesmo usuário atualiza a senha, não duplica a entrada.
<code>test_generated_credentials_actually_authenticate_against_mosquitto</code>	A senha gerada pelo script é de fato aceita por um Mosquitto real que carrega esse arquivo — confirmação ponta a ponta, não só "o arquivo tem o formato certo".

6.4 tests/test_docker_compose.py — docker-compose.yml real

Teste	O que verifica e por quê
<code>test_docker_compose_config_is_syntactically_valid</code>	<code>docker compose config</code> resolve o arquivo sem erro de sintaxe/schema.
<code>test_docker_compose_up_starts_a_working_broker</code>	<code>docker compose up</code> de verdade sobe um broker que aceita conexão na porta mapeada — diferente dos outros testes desta seção, que usam <code>docker run</code> direto com config copiada, este é o único que exercita o <code>docker-compose.yml</code> tal como ele é.

6.5 tests/test_end_to_end.py — integração ponta a ponta (mock publisher → broker → ingestão)

Teste	O que verifica e por quê
test_mock_publisher_message_reaches_ingestion_correctly_transformed	Automatiza a validação manual repetida em todo checkpoint: o mock publisher (código de produção) publica de verdade num broker real, e o <code>make_on_message</code> da ingestão (também código de produção) recebe e transforma a mensagem corretamente.
test_multiple_snapshots_all_reach_ingestion_in_order	Várias mensagens em sequência chegam todas, na ordem certa, sem perda nem duplicação.

7 O que ainda não é testado (decisão consciente, não descuido)

- `infra/mosquitto/mosquitto.conf` e `acl.conf` **em si** não têm teste unitário próprio — são exercitados indiretamente por *todos* os testes de `infra/`, já que o Mosquitto real só sobe se esses arquivos forem sintaticamente válidos.
- **Checkpoints 5 em diante** (InfluxDB, Grafana, firmware real) ainda não existem, então não há teste pra eles ainda — entram na mesma disciplina de TDD conforme cada checkpoint for aberto.
- `.github/workflows/ci.yml` não tem teste automatizado de si mesmo (não é comum nem prático testar um workflow de CI com outro teste) — a validação é rodar de verdade e observar o resultado, o que foi feito manualmente a cada mudança nele.